

# FactoryMethod e SOLID

Vamos analisar o projeto como um todo em relação aos princípios SOLID, considerando a estrutura e as responsabilidades das classes e interfaces envolvidas.

## Estrutura do Projeto

### 1. Interfaces e Entidades

- **IMobilia** : Interface que define as propriedades e métodos comuns a todas as mobílias.
- **IMobiliaFactory** : Interface que define o método **CreateMobilia** para criar objetos **IMobilia**.

### 2. Fábricas Concretas

- **SofaFactory** : Implementa **IMobiliaFactory** para criar objetos **Sofa**.
- **TableFactory** : Implementa **IMobiliaFactory** para criar objetos **Table**.
- **ChairFactory** : Implementa **IMobiliaFactory** para criar objetos **Chair**.
- **BedFactory** : Implementa **IMobiliaFactory** para criar objetos **Bed**.

### 3. Factory Creator

- **MobiliaFactoryCreator** : Classe que centraliza a criação das fábricas de mobília. Contém um dicionário que mapeia strings para instâncias de **IMobiliaFactory** e fornece o método **GetMobiliaFactoryByString** para obter a fábrica correta com base em uma string.

### 4. Modelo Principal

- **MainModel** : Classe que utiliza **MobiliaFactoryCreator** para converter uma lista de strings em uma lista de objetos **IMobilia** e delega a criação de relatórios para **ICreateMobiliaRelatory**.

### 5. Serviços

- **CreateMobiliaRelatory** : Classe que cria um relatório a partir de uma lista de objetos **IMobilia**, utilizando um **IRelatoryFormatter**.

### 6. Adaptadores

- **MobiliaAdapter** : Classe que converte uma lista de strings em uma lista de objetos **IMobilia** .

## 7. Apresentador

- **MainPresenter** : Classe que coordena a interação entre a visão ( **IMainView** ) e o modelo ( **IMainModel** ).

## 8. Visão

- **MainView** : Classe que gerencia a interface do usuário e interage com o usuário.

# Análise dos Princípios SOLID

## Princípio da Responsabilidade Única (SRP)

- **MainModel** : Coordena a conversão de listas de strings em listas de objetos **IMobilia** e a criação de relatórios. Está de acordo com o SRP, pois sua responsabilidade é a coordenação.
- **CreateMobiliaRelatory** : Cria um relatório a partir de uma lista de objetos **IMobilia** . Está de acordo com o SRP.
- **MobiliaAdapter** : Converte uma lista de strings em uma lista de objetos **IMobilia** . Está de acordo com o SRP.
- **MainPresenter** : Coordena a interação entre a visão e o modelo. Está de acordo com o SRP.
- **MainView** : Gerencia a interface do usuário e interage com o usuário. Está de acordo com o SRP.

## Princípio Aberto/Fechado (OCP)

- **MainModel** : Está aberta para extensão através das interfaces, sem necessidade de modificar a classe existente. Está de acordo com o OCP.
- **CreateMobiliaRelatory** : Está aberta para extensão através da interface **IRelatoryFormatter** . Está de acordo com o OCP.
- **MobiliaAdapter** : Está aberta para extensão através da interface **IMobiliaFactoryCreator** . Está de acordo com o OCP.
- **MainPresenter** : Está aberta para extensão através das interfaces **IMainView** e **IMainModel** . Está de acordo com o OCP.

- **MainView** : Está aberta para extensão através da interface **IMainView** . Está de acordo com o OCP.

## Princípio de Substituição de Liskov (LSP)

- **MainModel** : Implementa a interface **IMainModel** , permitindo substituição por qualquer implementação dessa interface. Está de acordo com o LSP.
- **CreateMobiliaRelatory** : Implementa a interface **ICreateMobiliaRelatory** , permitindo substituição por qualquer implementação dessa interface. Está de acordo com o LSP.
- **MobiliaAdapter** : Implementa a interface **IMobiliaAdapter** , permitindo substituição por qualquer implementação dessa interface. Está de acordo com o LSP.
- **MainPresenter** : Utiliza as interfaces **IMainView** e **IMainModel** , permitindo substituição por qualquer implementação dessas interfaces. Está de acordo com o LSP.
- **MainView** : Implementa a interface **IMainView** , permitindo substituição por qualquer implementação dessa interface. Está de acordo com o LSP.

## Princípio da Segregação de Interface (ISP)

- **IMainModel** : Define apenas os métodos necessários para a conversão e criação de relatórios. Está de acordo com o ISP.
- **ICreateMobiliaRelatory** : Define apenas os métodos necessários para a criação de relatórios. Está de acordo com o ISP.
- **IMobiliaAdapter** : Define apenas os métodos necessários para a conversão de listas de strings em listas de objetos **IMobilia** . Está de acordo com o ISP.
- **IMainView** : Define apenas os métodos necessários para a interação com a interface do usuário. Está de acordo com o ISP.

## Princípio da Inversão de Dependência (DIP)

- **MainModel** : Depende das abstrações **IMobiliaFactoryCreator** , **IMobiliaAdapter** e **ICreateMobiliaRelatory** , não de implementações concretas. Está de acordo com o DIP.
- **CreateMobiliaRelatory** : Depende da abstração **IRelatoryFormatter** , não de implementações concretas. Está de acordo com o DIP.

- **MobiliaAdapter** : Depende da abstração **IMobiliaFactoryCreator** , não de implementações concretas. Está de acordo com o DIP.
- **MainPresenter** : Depende das abstrações **IMainView** e **IMainModel** , não de implementações concretas. Está de acordo com o DIP.
- **MainView** : Depende da abstração **IMainView** , não de implementações concretas. Está de acordo com o DIP.

## Conclusão

O projeto está bem alinhado com os princípios SOLID. As classes e interfaces têm responsabilidades claras e bem definidas, estão abertas para extensão, permitem substituição por implementações intercambiáveis, definem apenas os métodos necessários e dependem de abstrações em vez de implementações concretas.